AWS DevOps Interview Master Notebook

Kubernetes — Lesson 1: Core Architecture, Pods, ReplicaSets & Scheduling

Purpose: Interview-ready notes with simple explanations, project-style answers, troubleshooting commands, scenarios, and key terms.

Preparation track: AWS DevOps → Kubernetes → Lesson-wise learning → Practice → Interview readiness

Status: Lesson 1 Master Notes

1. How to Use This Notebook

Use this chapter in three passes: Understand → Explain aloud → Troubleshoot from commands. The interview answers are deliberately written in a way that can be spoken naturally rather than memorized word-for-word.

Core interview rule: Always identify (1) what failed, (2) which Kubernetes component is responsible, (3) what object/state you inspect, and (4) which command proves your conclusion.

Lesson 1 Coverage

Topic	What you must know
Kubernetes architecture	Control plane, worker nodes, API server, etcd, scheduler, controller manager, kubelet, contain
Pod	Smallest deployable unit, containers, lifecycle, restart behavior
ReplicaSet	Maintains desired number of Pod replicas and recreates failed Pods
Scheduler	Finds an appropriate node for a newly created unscheduled Pod
Controllers	Continuously reconcile desired state with actual state
CrashLoopBackOff	Meaning, common causes, diagnosis
Troubleshooting	get, describe, logs, events, rollout/status checks
Interview scenarios	Pod crash, unscheduled Pod, ReplicaSet failure, node issues

2. Kubernetes Architecture — The Big Picture

Kubernetes is a container orchestration platform. Its job is to keep applications running according to the desired state you declare. You usually say what you want — for example, 3 replicas of an application — and Kubernetes continuously works to make reality match that desired state.

Think of the cluster as two major parts:

Part	Simple meaning	Main responsibility
Control Plane	The brain/management layer	Accepts requests, stores cluster state, schedules Pods, and runs c
Worker Node	The machine that runs workloads	Runs Pods and reports their status back to the control plane

2.1 Control Plane Components

kube-apiserver: The front door of the cluster. kubectl communicates with the API server. Kubernetes components also use the API server to read or update cluster state.

etcd: A distributed key-value store that holds Kubernetes cluster state and configuration. It is the persistent source of truth for the control plane.

kube-scheduler: Watches for newly created Pods that do not yet have a node assigned and selects a suitable worker node.

kube-controller-manager: Runs controllers that continuously compare desired state with actual state and take corrective action.

cloud-controller-manager: In cloud environments, integrates Kubernetes with cloud-provider functionality such as nodes, load balancers, and routes, depending on the provider/setup.

2.2 Worker Node Components

kubelet: The agent on each worker node. It receives Pod-related instructions through the Kubernetes control plane and makes sure the containers described by Pods are running.

Container runtime: The software that actually runs containers. Kubernetes commonly uses a CRI-compatible runtime such as containerd.

kube-proxy: Implements network service routing behavior on nodes in traditional Kubernetes networking setups. Modern clusters may use alternative networking/data-plane implementations.

2.3 Easy Architecture Flow to Remember

```
You → kubectl
↓
kube-apiserver
↓
etcd
↓
Controllers / Scheduler
↓
Worker Node
↓
kubelet
↓
Container runtime
↓
Pod
```

> Interview sentence: “The API server is the entry point, etcd stores cluster state, the scheduler places unscheduled Pods, controllers maintain desired state, and worker-node components actually run and manage the containers.”

3. Key Terms You Must Not Mix Up

Term	Definition	Remember it as
Cluster	A group of machines managed by Kubernetes.	Whole Kubernetes environment
Control plane	Management components that make cluster-level decisions.	Brain
Worker node	Machine where application Pods run.	Work machine
Pod	Smallest deployable unit in Kubernetes; contains one or more containers.	Wrapper around containers
Container	An isolated application process packaged with its dependencies.	Actual app process
ReplicaSet	Controller that maintains a specified number of matching Pod replicas.	Keep N Pods alive
Deployment	Higher-level controller that manages ReplicaSets and supports declarative app updates.	Manage app versions
Desired state	The state you declare, such as replicas: 3.	What I want
Actual state	What is currently running in the cluster.	What exists now
Reconciliation	The continuous process of moving actual state toward desired state.	Make reality match config
Scheduler	Component that chooses a node for an unscheduled Pod.	Where should Pod run?
kubelet	Node agent responsible for making sure assigned Pods are running.	Node worker agent
etcd	Persistent distributed key-value store for Kubernetes state.	Cluster source of truth
API server	Central Kubernetes API endpoint.	Front door

4. Pod — The Most Important Basic Object

A Pod is the smallest deployable unit in Kubernetes. Kubernetes schedules Pods onto nodes; it does not normally schedule individual containers directly.

A Pod can contain one or more containers. Containers inside the same Pod share the Pod's network namespace and can share mounted storage volumes. A common application design has one main application container per Pod, with additional sidecars only when needed.

Example: If you declare 3 replicas of an nginx application, Kubernetes aims to have 3 Pods running that application.

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: web
spec:
replicas: 3
selector:
matchLabels:
app: web
template:
metadata:
labels:
app: web
spec:
containers:
- name: nginx
image: nginx:latest
```

This Deployment creates/manages a ReplicaSet, which in turn creates and maintains the Pods.

4.1 Pod Lifecycle — Interview View

Typical Pod phases include Pending, Running, Succeeded, Failed, and Unknown. Be careful: a Pod phase is not the same thing as a container state such as Waiting, Running, or Terminated.

Pending: Pod has been accepted but is not yet fully running. It may be waiting for scheduling, image pull, or other setup.

Running: Pod has been bound to a node and at least one container is running or starting.

Succeeded: All containers terminated successfully.

Failed: All containers terminated and at least one failed.

Unknown: Kubernetes cannot obtain the Pod's state, often due to communication problems with the node.

5. ReplicaSet — Who Replaces a Crashed Pod?

A ReplicaSet is a Kubernetes controller whose job is to maintain a desired number of matching Pod replicas.

Suppose the desired state is 3 Pods. If 3 are running, the ReplicaSet is satisfied. If one Pod disappears or is deleted and only 2 remain, the ReplicaSet notices that actual state is 2 while desired state is 3, so it creates another matching Pod.

```
Desired: 3 Pods
Actual: 3 Pods
Action: Nothing

One Pod crashes/is removed:
Desired: 3
Actual: 2
↓
ReplicaSet reconciles
↓
Creates replacement Pod
↓
```

```
Actual returns to 3
```

Critical clarification: A ReplicaSet does not “repair the crashed process inside the same Pod.” It maintains the replica count. Depending on the failure, the kubelet/container runtime may restart a container within the Pod, while the ReplicaSet creates a replacement Pod when the Pod itself is no longer present/available to satisfy the controller's desired replica count.

Deployment relationship: In most application deployments, you create a Deployment rather than a ReplicaSet directly. The Deployment manages ReplicaSets, and the ReplicaSet manages Pods.

```
Deployment
↓
ReplicaSet
↓
Pods
↓
Containers
```

6. Scheduler — Who Assigns a New Pod to a Node?

The kube-scheduler watches for newly created Pods that do not have a node assigned. It evaluates eligible nodes and selects one based on scheduling rules and constraints.

Examples of factors include resource requests, node selectors, affinity/anti-affinity, taints and tolerations, topology constraints, and other scheduling policies.

Important distinction: The scheduler decides which node should run the Pod. The kubelet on that node is responsible for making the assigned Pod actually run.

```
Pod created
↓
API Server
↓
Scheduler sees unscheduled Pod
↓
Finds suitable node
↓
Pod gets node assignment
↓
kubelet on that node starts/manages containers
```

6.1 Scenario: Pod Created but Not Assigned to Any Node

Question: “A new Pod is created but isn't assigned to any node. Which component is responsible?”

> Answer: “The kube-scheduler is responsible for selecting a suitable node for an unscheduled Pod. I would check the Pod status and events to understand why scheduling has not happened.”

```
kubectl get pod -o wide
kubectl describe pod
kubectl get events --sort-by=.lastTimestamp
```

In kubectl describe pod, inspect the Events section. Common reasons include insufficient CPU/memory, node selector mismatch, affinity rules, taints without matching tolerations, or lack of suitable nodes.

7. CrashLoopBackOff — What It Actually Means

CrashLoopBackOff means a container has repeatedly started, crashed, and Kubernetes is backing off — waiting progressively longer before attempting another restart.

It is not itself the root cause. It is a symptom/status indicating repeated container failures.

Common causes: application exception, wrong command/entrypoint, missing environment variable, missing Secret/ConfigMap value, failed dependency connection, permission problem, bad configuration, or an application that exits immediately.

> Interview answer: “I would not simply restart the Pod. First I would identify why the container is exiting by checking the current logs, previous container logs, Pod description, and events.”

```
kubectl get pod
kubectl describe pod
kubectl logs -c
kubectl logs -c --previous
kubectl get events --sort-by=.lastTimestamp
```

Why --previous? When a container has crashed and restarted, the previous instance's logs may contain the error that caused the crash. This is often one of the most useful commands for CrashLoopBackOff.

8. Scenario-Based Interview Questions

Scenario 1 — A Pod Crashes. Who Replaces It?

Question: “A Pod crashes. Who replaces it?”

> Strong answer: “It depends on what exactly failed. The kubelet is responsible for managing containers in a Pod and can restart a failed container according to the Pod's restart policy. If the Pod itself is deleted or no longer exists and it is managed by a ReplicaSet, the ReplicaSet controller detects that the actual replica count is below the desired count and creates a replacement Pod. If the application is managed by a Deployment, the Deployment manages the ReplicaSet, which manages the Pods.”

Follow-up: “What would you check?”

```
kubectl get pod
kubectl describe pod
kubectl logs
kubectl logs --previous
kubectl get rs
kubectl describe rs
kubectl get events --sort-by=.lastTimestamp
```

Do not say: “ReplicaSet restarts the crashed container.” Better: “kubelet/container runtime handles container restarts; ReplicaSet maintains the desired number of Pods.”

Scenario 2 — New Pod Has No Node

Question: “A new Pod is created but isn't assigned to any node. Which component is responsible?”

> Strong answer: “The kube-scheduler. I would run kubectl get pod -o wide and kubectl describe pod. If the Pod remains Pending, I would inspect the Events section for scheduling failures such as insufficient resources, taints, node selectors, affinity rules, or other constraints.”

Scenario 3 — Desired Replicas = 3, Only 2 Exist

Question: “A ReplicaSet wants 3 Pods but only 2 are running. What happens?”

> Answer: “The ReplicaSet controller continuously reconciles desired and actual state. It detects that one replica is missing and creates another Pod. The scheduler then assigns that new Pod to a suitable node, and the kubelet starts its containers.”

```
ReplicaSet notices: 3 desired vs 2 actual
↓
Creates Pod
```

```
↓
Scheduler assigns node
↓
Kubelet starts Pod
```

Scenario 4 — Pod Is Pending

Question: “A Pod is stuck in Pending. What do you do?”

Answer: “First I determine whether it is waiting for scheduling or another startup step. I check kubectl describe pod and events. I verify node availability, CPU/memory requests, node selectors, affinity, taints/tolerations, and image-related messages. If it has been scheduled but is still Pending, I inspect the specific container and volume/image events.”

9. Essential kubectl Commands for Lesson 1

Command	Purpose
kubectl get pods	List Pods in the current namespace.
kubectl get pods -o wide	Show Pod IP and node information too.
kubectl describe pod <pod>	Detailed Pod information plus Events — essential for troubleshooting.
kubectl logs <pod>	View container logs.
kubectl logs <pod> --previous	View logs from the previous crashed container instance.
kubectl get rs	List ReplicaSets.
kubectl describe rs <rs>	Inspect ReplicaSet details and events.
kubectl get deployment	List Deployments.
kubectl get nodes	List worker/control-plane nodes visible to the cluster.
kubectl describe node <node>	Inspect node conditions, resources, taints, and events.
kubectl get events --sort-by=.lastTimestamp	Review recent cluster events in time order.

10. High-Probability Interview Question Bank

Q1. What is Kubernetes?

Kubernetes is an open-source container orchestration platform used to deploy, manage, scale, and maintain containerized applications. It provides declarative configuration and continuously works to keep the cluster in the desired state.

Q2. What is a Kubernetes cluster?

A cluster is the complete Kubernetes environment consisting of the control plane and worker nodes.

Q3. What is the control plane?

The control plane contains the components that manage cluster state and make cluster-level decisions, including the API server, etcd, scheduler, and controllers.

Q4. What is a worker node?

A worker node is a machine that runs application workloads. It typically has kubelet, a container runtime, and networking components.

Q5. What is a Pod?

A Pod is the smallest deployable unit in Kubernetes and contains one or more containers that share networking and can share storage.

Q6. Why does Kubernetes use Pods instead of scheduling containers directly?

The Pod provides a common execution and networking context for one or more closely coupled containers. Kubernetes schedules the Pod as the unit of deployment.

Q7. What is a ReplicaSet?

A ReplicaSet is a controller that maintains a desired number of matching Pod replicas.

Q8. Who replaces a deleted Pod?

If the Pod is managed by a ReplicaSet/Deployment, the ReplicaSet controller detects the replica deficit and creates a replacement. The scheduler then places it on a node and the kubelet runs it.

Q9. Who restarts a crashed container?

The kubelet manages containers in the Pod and, according to the Pod's restart policy, works with the container runtime to restart failed containers.

Q10. What is kube-scheduler?

It selects a suitable node for a newly created Pod that does not yet have a node assignment.

Q11. What is kubelet?

The kubelet is the node agent that ensures Pods assigned to its node are running and healthy according to their specifications.

Q12. What is etcd?

etcd is a distributed key-value store that persistently stores Kubernetes cluster state.

Q13. What does the API server do?

It exposes the Kubernetes API and acts as the central entry point through which kubectl and Kubernetes components interact with cluster state.

Q14. What is desired state?

The state declared by the user, such as replicas: 3.

Q15. What is actual state?

The state currently observed in the cluster, such as only 2 matching Pods running.

Q16. What is reconciliation?

Controllers continuously compare desired state with actual state and take actions to reduce the difference.

Q17. What is CrashLoopBackOff?

A status indicating that a container has repeatedly failed and Kubernetes is applying a backoff delay between restart attempts.

Q18. How do you troubleshoot CrashLoopBackOff?

Check describe output, current logs, previous logs, and events; then investigate configuration, command, dependencies, permissions, probes, and resources.

Q19. A Pod is Pending. What do you check?

Start with kubectl describe pod and Events. Check scheduling constraints, node availability, resources, taints/tolerations, selectors, affinity, and any image/volume errors.

Q20. Scheduler vs kubelet?

Scheduler decides where a Pod should run. Kubelet on that node makes sure the assigned Pod's containers actually run.

Q21. ReplicaSet vs Deployment?

A ReplicaSet maintains a set number of Pods. A Deployment is a higher-level controller that manages ReplicaSets and provides declarative application updates and rollouts.

11. Mini Practical Exercise

Create a Deployment with 3 replicas and observe the controller chain.

```
kubectl create deployment web --image=nginx --replicas=3
kubectl get deployment
kubectl get rs
kubectl get pods -o wide
```

Now delete one Pod and watch Kubernetes recover the desired replica count.

```
kubectl delete pod
kubectl get pods -w
kubectl get rs
```

What you should observe: the deleted Pod disappears; the ReplicaSet detects fewer replicas than desired and creates a replacement; the scheduler assigns the new Pod to a node; the kubelet starts its container.

12. Troubleshooting Decision Tree

```
Problem: Pod is not healthy
|
+-- Is Pod Pending?
| |
| +-- describe pod → Events
| → scheduler/resources/taints/selectors/etc.
|
+-- Is Pod CrashLoopBackOff?
| |
| +-- logs
| +-- logs --previous
| +-- describe pod → Events
| → app/config/command/dependency/probe/etc.
|
+-- Pod disappeared?
|
+-- Check Deployment / ReplicaSet
→ desired replicas vs actual Pods
→ replacement should be created
```

13. One-Minute Revision Sheet

Kubernetes = desired state management for containers.

API Server = front door.

etcd = persistent cluster state.

Scheduler = decides which node.

Kubelet = makes assigned Pods run on the node.

Pod = smallest deployable unit.

ReplicaSet = maintains desired Pod count.

Deployment = manages ReplicaSets and application rollouts.

CrashLoopBackOff = container repeatedly crashes; investigate logs/events.

Pending = Pod is not fully running; describe it and inspect Events.

Desired state ≠ actual state → controllers reconcile.

14. Interview Answer Formula

For almost every Kubernetes scenario, answer using this pattern:

1. Identify the symptom.
2. Name the responsible component.
3. Explain what that component does.
4. Give 2–4 kubectl commands.
5. Mention what evidence you would inspect.
6. State the likely root-cause categories.

Example: “The Pod is Pending. The scheduler is responsible for node placement. I would run kubectl describe pod and inspect Events, then check resources, taints/tolerations, selectors and affinity. If it is scheduled but containers are not starting, I would move on to image, volume and container-level troubleshooting.”

15. Common Interview Traps

• Trap: “ReplicaSet restarts containers.” → Correct: kubelet/runtime handles container restart; ReplicaSet maintains Pod replicas.

• Trap: “Scheduler starts the container.” → Correct: scheduler selects a node; kubelet/runtime run the containers.

• Trap: “CrashLoopBackOff is the root cause.” → Correct: it signals repeated crashes; logs/events reveal the cause.

• Trap: “Pod = container.” → Correct: Pod is the Kubernetes deployment unit and can contain one or more containers.

• Trap: “Pending always means scheduler problem.” → Correct: scheduling is a common reason, but Pending can also involve image pulls, volumes, initialization, and other startup conditions; inspect Events.

16. Lesson 1 Interview-Readiness Checklist

Skill	Target
Explain control plane vs worker node without notes	☐
Explain API server, etcd, scheduler, controller manager	☐
Explain Pod in simple words	☐
Explain Deployment → ReplicaSet → Pod relationship	☐
Answer “Who replaces a Pod?” accurately	☐
Answer “Who assigns Pod to node?” accurately	☐
Explain scheduler vs kubelet	☐
Explain CrashLoopBackOff and use --previous logs	☐
Use describe + Events for Pending Pods	☐
Run the mini practical exercise	☐

Skill	Target
Answer all 21 interview questions aloud	☐

Next notebook section can extend this same master PDF with Lesson 2 and later Kubernetes topics while preserving the same interview-question + scenario + command + project-answer structure.